EXPERIMENT 1

AI-Assisted Boolean Expression Simplification and Optimization

Curriculum: Part B · GenAI-assisted design **Course outcome:** CO1, CO2 **EDA environment:** Logisim / Logisim Evolution

1.1 Aim and objectives

To reduce a four-input Boolean function to a compact Sum-of-Products (SOP) form with the help of Karnaugh-map grouping and AI-assisted analysis. The reduced expression is then translated into a gate-level circuit and checked against the original function.

1.2 Background

Boolean minimization removes unnecessary literals and repeated logic conditions. A Karnaugh map arranges minterms so that adjacent 1-cells can be combined in groups of 1, 2, 4 or 8. Each valid group eliminates variables that change within that group. A smaller expression generally needs fewer gates and interconnections, which can simplify implementation and reduce loading and delay.

The function supplied for this experiment is F(A,B,C,D) = Σm(0, 2, 8, 10, 12, 14), where A is the most significant input and D is the least significant input.

1.3 AI-assisted approach

The prompt asked the AI assistant to place the listed minterms in a four-variable K-map, identify useful groups (including wrap-around adjacency), and produce the simplest equivalent SOP expression. The result was checked against the K-map rather than accepted without verification.



Figure 1.1. Source AI-assistant output used during the minimization exercise.

1.4 K-map grouping and minimized expression

Two four-cell groups cover the six required minterms. The corner group (0, 2, 8, 10) leaves B = 0 and D = 0 unchanged, producing B′D′. The second group (8, 10, 12, 14) keeps A = 1 and D = 0, producing AD′. Minterms 8 and 10 are shared by both groups, which is permitted and ensures complete coverage.

F = B′D′ + AD′ = D′(B′ + A)

1.5 Hardware comparison

The source portfolio compares a canonical implementation with a minimized implementation. The key design improvement is the reduction in the number of product terms and the removal of dependence on C. Exact gate totals depend on whether multi-input gates and shared inversions are counted as separate components.

DESIGN ASPECT	CANONICAL FORM	MINIMIZED FORM
Logic structure	Six minterms combined by an OR stage	Two product terms combined by an OR stage
Input dependence	A, B, C and D appear in minterms	C is eliminated; D is complemented
Implementation impact	More gates and wiring	Smaller network and simpler routing
Expected timing	More logic stages / loading	Shorter, simpler logic path

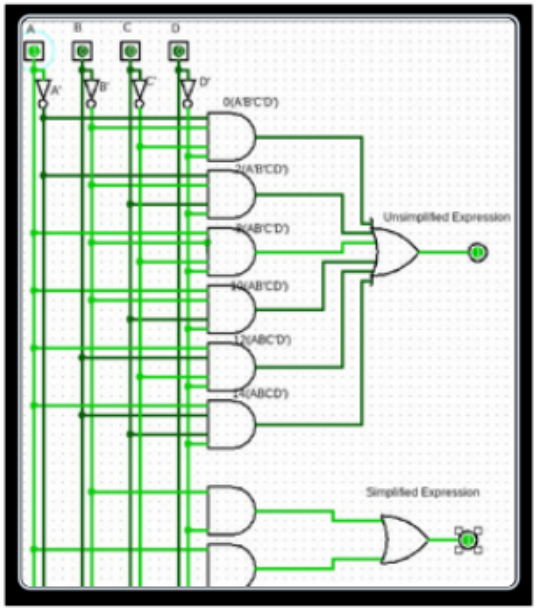


Figure 1.2. Original Logisim comparison of the unsimplified and minimized implementations.

1.6 Verification and observation

The truth table contains sixteen input combinations. The function is HIGH for minterms 0, 2, 8, 10, 12 and 14, and LOW for all remaining combinations. The minimized equation produces the same output set, so it is logically equivalent to the original specification.

INPUT MINTERMS	EXPECTED F	CHECK
0, 2, 8, 10, 12, 14	1	Covered by the two K-map quads
1, 3, 4, 5, 6, 7, 9, 11, 13, 15	0	No implicant is active

1.7 Result and conclusion

The K-map method reduced the supplied function to F = B′D′ + AD′. The circuit screenshot and truth-table comparison document the implementation check. This experiment illustrates how AI can assist with grouping and explanation, while the designer remains responsible for checking every implicant and output combination.

EXPERIMENT 2

Natural Language Processing-Based Logic Circuit Generation

Curriculum: Part B · GenAI-assisted design **Course outcome:** CO2 **EDA environment:** Logisim / Logisim Evolution

2.1 Aim

To translate a written security requirement into Boolean logic, select an efficient gate arrangement, and validate the resulting authorization circuit in Logisim.

2.2 Design statement and interpretation

The proposed vault opens only when the master key is present and at least one of the two secondary credentials is valid. This means the key condition is mandatory, while the manager PIN and biometric scan are alternative ways to satisfy the second condition.

SIGNAL	SYMBOL	LOGIC 0	LOGIC 1	POSSIBLE SENSOR
Master key	A	Key absent	Key inserted and turned	Key-switch sensor
Manager PIN	B	PIN invalid	PIN valid	Keypad / PIN comparator
Biometric	C	Scan rejected	Identity verified	Fingerprint scanner
Vault release	Y	Locked	Open	Solenoid driver

2.3 Boolean translation

The phrase “PIN is correct OR biometric scan is successful” becomes (B + C). The word “AND” connects this combined condition to the master-key input A.

$$Y = A(B + C)$$

2.4 Gate-level design

The factored expression can be built using one OR gate followed by one AND gate. An expanded SOP form, AB + AC, is also valid but typically requires two AND gates and one OR gate. The factored form therefore provides a compact implementation for this requirement.

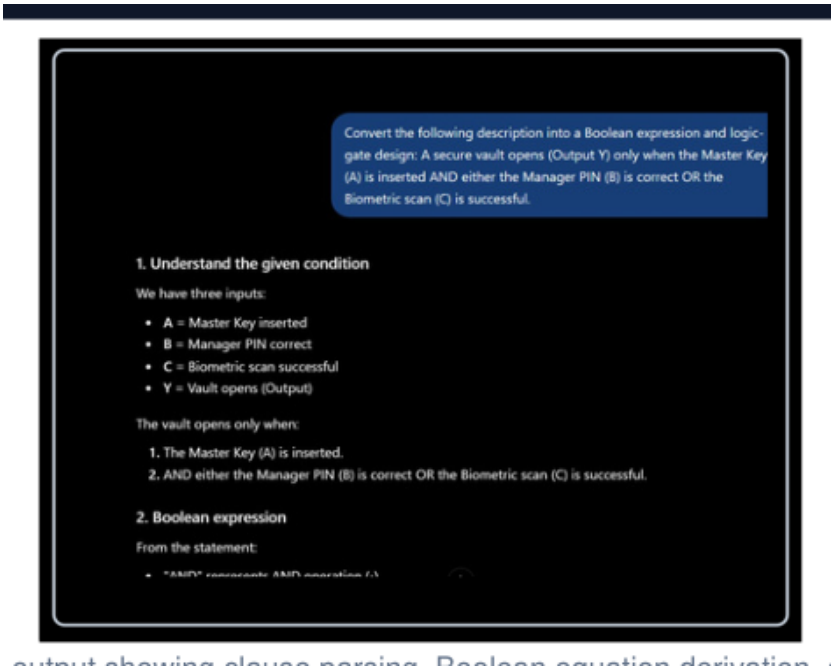


Figure 2.1. Source AI response showing the conversion from the written requirement to Boolean logic.

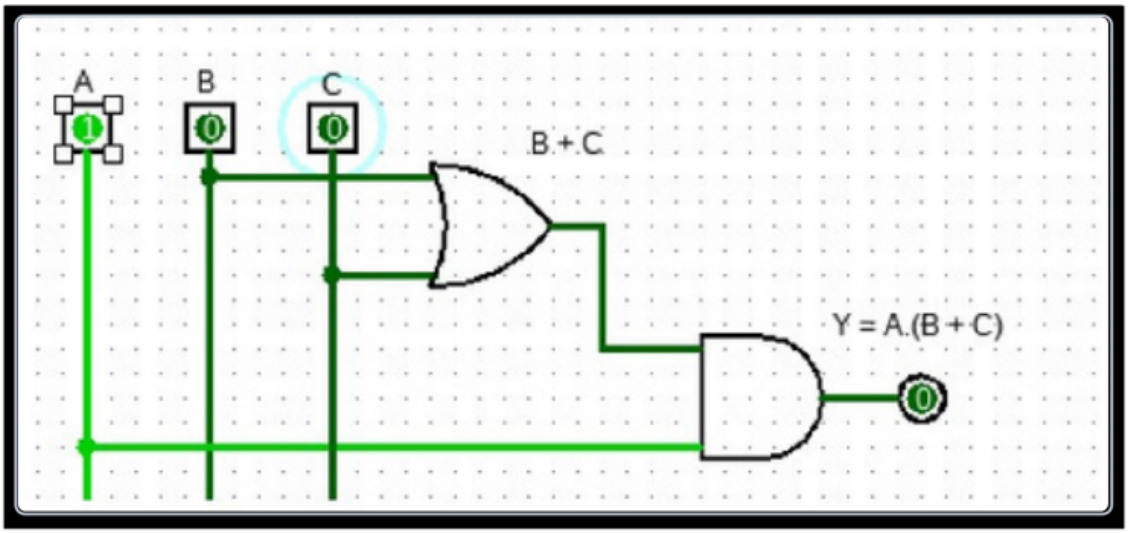


Figure 2.2. Logisim implementation of the factored vault-release expression.

2.5 Functional verification

The output can be HIGH only in the four cases where A = 1 and at least one of B or C is HIGH. If A = 0, the vault remains locked regardless of the secondary credentials.

A	B	C	B + C	Y = A(B + C)	INTERPRETATION
0	0	0	0	0	No key; locked
0	0	1	1	0	Biometric alone is insufficient
0	1	0	1	0	PIN alone is insufficient
0	1	1	1	0	No master key
1	0	0	0	0	Key only; locked
1	0	1	1	1	Key + biometric

A	B	C	B + C	Y = A(B + C)	INTERPRETATION
1	1	0	1	1	Key + PIN
1	1	1	1	1	Both credentials accepted

2.6 Result and conclusion

The English-language specification was represented by Y = A(B + C). The gate arrangement and all eight truth-table cases support the stated access rule. The exercise demonstrates a repeatable workflow: identify conditions, assign variables, translate connectives, simplify, implement, and verify.

EXPERIMENT 3

FSM-Based Traffic Light Controller Design Using Generative AI

Curriculum: Part B · GenAI-assisted design **Course outcome:** CO2, CO3 **EDA environment:** Logisim / Logisim Evolution

3.1 Aim

To construct a synchronous Moore finite-state machine (FSM) for a three-colour traffic signal. A pedestrian request initiates the sequence Green → Yellow → Red → Green.

3.2 Moore-machine specification

The controller has one input P and three outputs G, Y and R. Outputs depend on the present state, not directly on P. Two D flip-flops encode the three active states; the unused encoding 11 is treated as an unused/recovery state in the supplied design.

STATE	Q1 Q0	OUTPUT	TRANSITION
S0 – Green	00	G=1, Y=0, R=0	Remain in S0 if P=0; move to S1 if P=1
S1 – Yellow	01	G=0, Y=1, R=0	Move to S2 on the next active clock
S2 – Red	10	G=0, Y=0, R=1	Move to S0 on the next active clock
S3 – Unused	11	All outputs 0	Recovery behaviour should be defined as S0

3.3 State-transition and D-input table

For D flip-flops, the next-state bits are applied at the next active clock edge. The table below lists the operational transitions.

PRESENT STATE	P	NEXT STATE	D1	D0
00 (Green)	0	00 (Green)	0	0
00 (Green)	1	01 (Yellow)	0	1
01 (Yellow)	X	10 (Red)	1	0
10 (Red)	X	00 (Green)	0	0
11 (Unused)	X	00 (Recovery)	0	0

$$D1 = Q0$$
$$D0 = Q1' \, Q0' \, P$$
$$G = Q1' \, Q0' \quad Y = Q1' \, Q0 \quad R = Q1 \, Q0'$$

3.4 AI prompt and implementation

The AI prompt requested a state table and D flip-flop equations for the specified Green, Yellow and Red sequence. The equations were then mapped to two state flip-flops and output-decoding logic in Logisim.

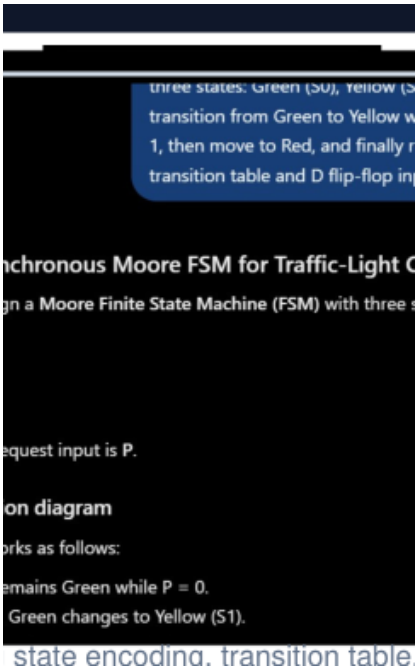


Figure 3.1. AI-generated state assignment and next-state logic from the source portfolio.

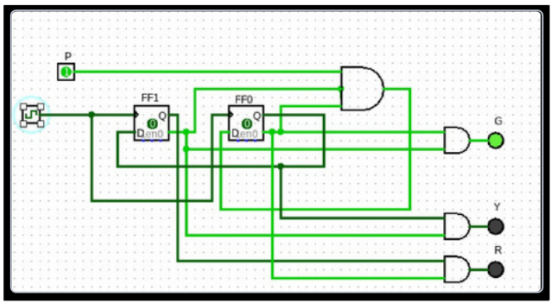


Figure 3.2: State S0 (Q1=0, Q0=0) · Green LED ON (G=1). Pedestrian button P=1 primed.

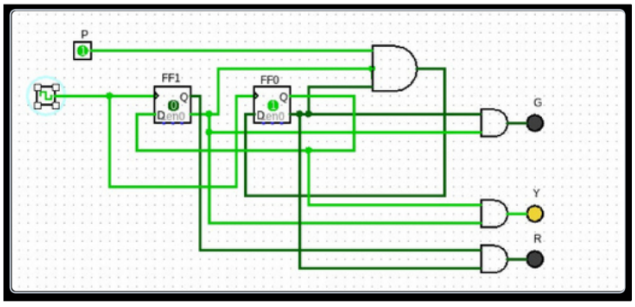


Figure 3.3: State S1 (Q1=0, Q0=1) · Yellow LED ON (Y=1). Next state logic primes D1=Q0=1 for Re

Figure 3.2. Original Logisim state-circuit screenshots for the traffic-light sequence.

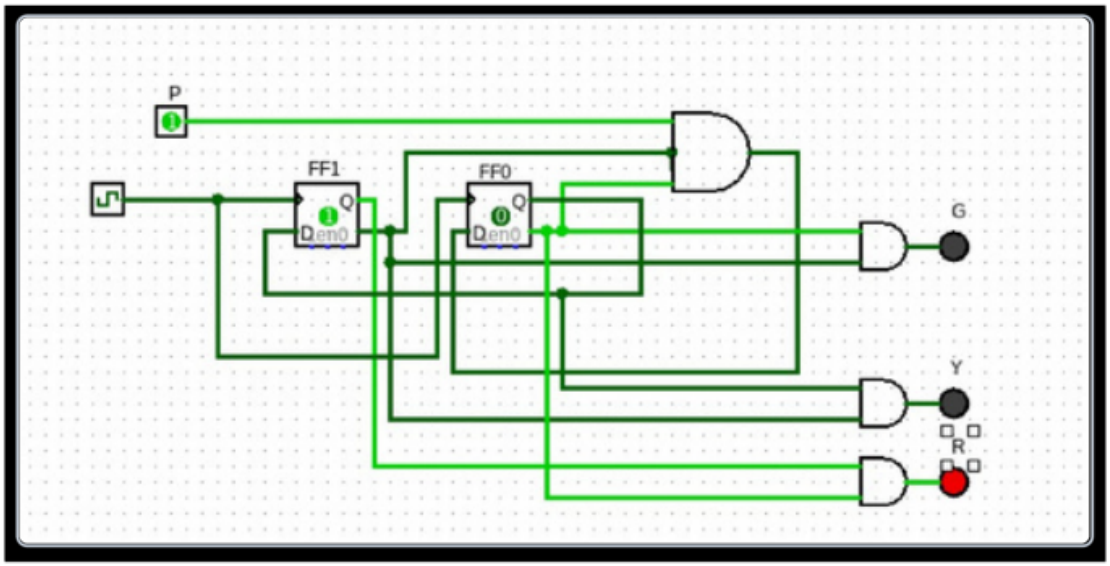


Figure 3.3. Additional source simulation view associated with the controller.

3.5 Timing sequence and checks

A request is sampled while the controller is in Green. The state changes on the next clock edge, after which the FSM advances automatically through Yellow and Red. The request input does not alter the outputs while the machine is already in Yellow or Red.

CLOCK EVENT	P	STATE	ACTIVE LIGHT	EXPECTED ACTION
Initial	0	00	Green	Normal traffic flow
Request sampled	1	00	Green	Next state is prepared as Yellow
Next edge	X	01	Yellow	Caution interval
Next edge	X	10	Red	Vehicles stop; pedestrian crossing phase
Next edge	0	00	Green	Normal sequence resumes

3.6 Result and conclusion

The two-bit Moore FSM represents the required three-state traffic-light cycle. The derived next-state and output equations provide a compact implementation. Simulation should confirm that state changes occur only at clock edges and that each state activates only its assigned light.

EXPERIMENT 4

AI-Based Fault Detection in Digital Circuits

Curriculum: Part B · GenAI-assisted design **Course outcome:** CO1, CO2 **EDA environment:** Logisim / Logisim Evolution

4.1 Aim

To detect a deliberate error in a one-bit full-adder circuit by comparing its measured truth table with the expected arithmetic behaviour, then repair the faulty Sum path.

4.2 Full-adder principle

A full adder accepts A, B and carry-in (Cin). Its Sum output is the least-significant bit of the three-input total, while Cout is the carry bit.

$$\text{Sum} = A \oplus B \oplus Cin$$
$$\text{Cout} = AB + ACin + BCin = AB + Cin(A \oplus B)$$

4.3 Fault scenario and AI diagnosis

The source assignment describes a missing Cin connection in the Sum network. However, its listed faulty output sequence for inputs 000 through 111 is 0, 0, 1, 1, 1, 1, 0, 0. That sequence matches A ⊕ B, meaning the Sum path behaves like a two-input XOR and ignores Cin. It does not match A·B. This distinction is important when localizing the fault.

$$\text{Observed faulty Sum: } A \oplus B$$
$$\text{Correct Sum: } A \oplus B \oplus Cin$$
$$\text{Correction: cascade a second XOR gate and connect Cin to it.}$$



Figure 4.1: GenAI truth table discrepancy analysis and fault localizati

Figure 4.1. Source AI diagnostic interaction.



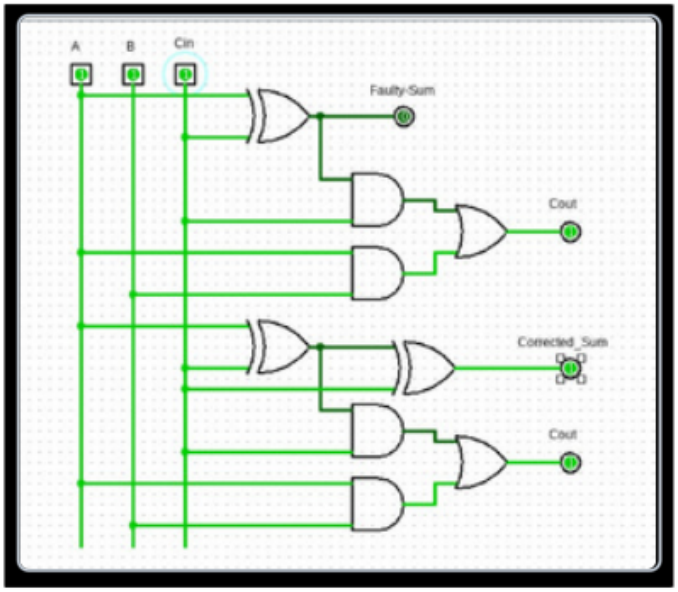
2: GenAI identification of Half Adder behavior and prescription of secon

Figure 4.2. Source truth-table evidence associated with the fault analysis.

4.4 Truth-table comparison

The expected Sum is the parity of the three inputs. Cout is HIGH when at least two inputs are HIGH.

A	B	Cin	FAULTY SUM (A⊕B)	CORRECT SUM	Cout
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	1	0	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	0	0	1
1	1	1	0	1	1



der test vector A=1, B=1, Cin=1. Top: Faulty Sum = 0 (Inc

Figure 4.3. Original circuit view from the source report showing the full-adder implementation.

4.5 Test case analysis

For A = 1, B = 1 and Cin = 1, the arithmetic total is 3, or binary 11. Therefore, both Sum and Cout must be 1. The faulty XOR-only output produces Sum = 0 for this input, while the corrected three-input parity expression produces Sum = 1.

4.6 Result and conclusion

The listed faulty vector identifies a Sum circuit that omits Cin and computes A ⊕ B. Adding a second XOR stage restores the correct full-adder Sum function. The carry expression must also be independently checked; a correct Cout alone does not guarantee a correct full-adder.

Verification note: The original source text contains a conflicting statement describing the faulty Sum as A·B. The explicit eight-value sequence is used here because it is consistent with the Boolean function A ⊕ B.

EXPERIMENT 5

24-Hour Digital Clock Using Finite State Machines

Curriculum: Part B · GenAI-assisted design **Course outcome:** CO2, CO3 **EDA environment:** Logisim / Logisim Evolution

5.1 Aim

To describe a 24-hour clock using interconnected synchronous counters for seconds, minutes and hours, and to examine rollover behaviour at each limit.

5.2 Theory and operating principle

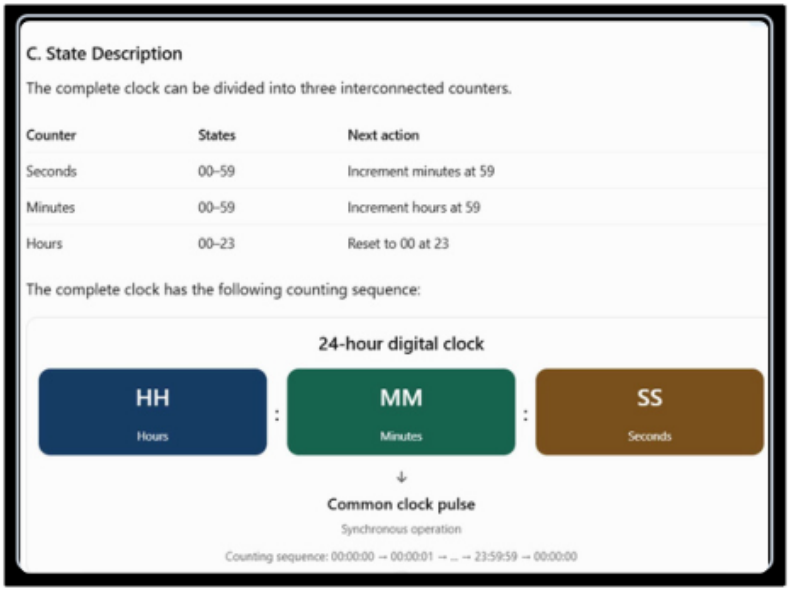
The clock advances once per second. The seconds field counts from 00 through 59; on the next pulse it returns to 00 and enables the minutes field to advance. Minutes behave in the same way. The hour field counts from 00 to 23 and then returns to 00, completing a 24-hour cycle.

COUNTER	VALID RANGE	ROLLOVER ACTION
Seconds	00–59	Return to 00 and increment minutes
Minutes	00–59	Return to 00 and increment hours
Hours	00–23	Return to 00 after 23

5.3 State progression examples

A common clock pulse is shared by the counter stages. Carry/enable signals ensure that a higher-order field changes only when the lower-order field reaches its terminal count.

CURRENT TIME	AFTER ONE SECOND PULSE
00:00:00	00:00:01
00:00:58	00:00:59
00:00:59	00:01:00
00:59:59	01:00:00
12:59:59	13:00:00
23:59:58	23:59:59
23:59:59	00:00:00



digital clock block diagram showing synchronous operation from C

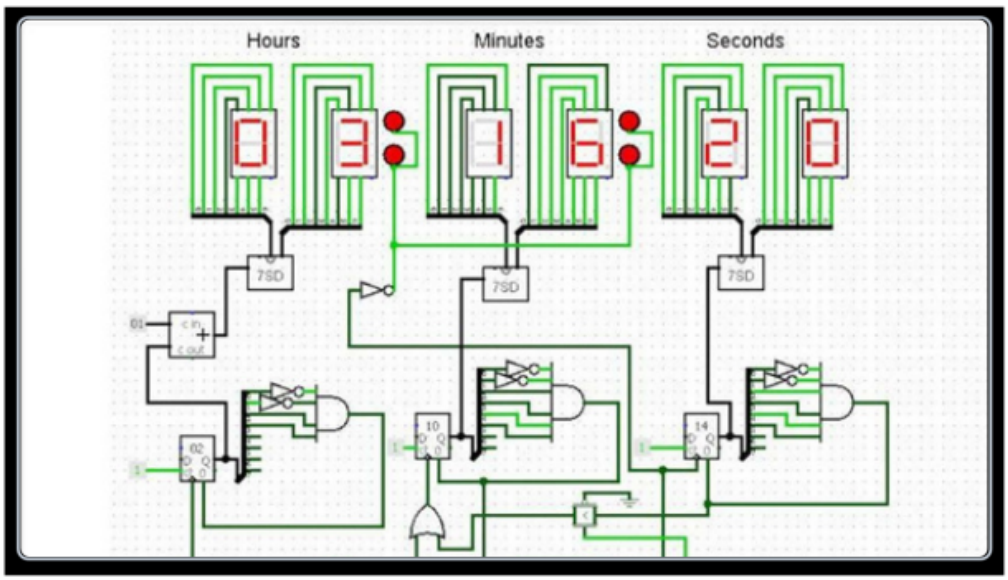
Figure 5.1. Source block-level view of the 24-hour clock and its display arrangement.

5.4 D flip-flop increment logic

For a four-bit binary incrementer with present-state bits Q3 Q2 Q1 Q0, the next-state equations for an enabled increment are:

$$D0 = Q0'$$
$$D1 = Q1 \oplus Q0$$
$$D2 = Q2 \oplus (Q1Q0)$$
$$D3 = Q3 \oplus (Q2Q1Q0)$$

These equations describe a binary increment stage. A practical clock must additionally decode the required terminal values (59 and 23) and implement the appropriate rollover and carry enables. The supplied portfolio presents the increment equations in a compact form; the XOR form above makes the bit-toggle conditions explicit.



re 5.2: Complete 24-hour digital clock Logisim simulation schematic with 7-segment di

Figure 5.2. Source Logisim circuit screenshot for the digital clock.

5.5 Result and conclusion

The clock architecture is formed from three linked counting sections. Correct operation is demonstrated by normal increments and the critical transitions 00:00:59 → 00:01:00, 00:59:59 → 01:00:00 and 23:59:59 → 00:00:00. The terminal-count logic is essential to keep the display within the 24-hour range.

EXPERIMENT 6

Application Mapping of Sequential Building Blocks

Curriculum: Part C · Real-world applications **Course outcome:** CO3, CO1 **EDA environment:** Logisim Evolution

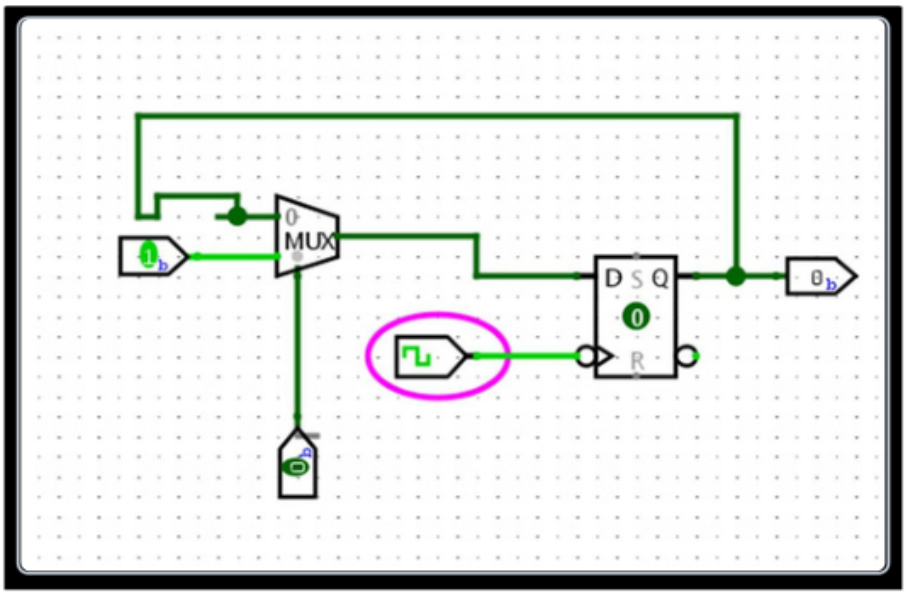
6.1 Aim

To examine three commonly used sequential components—a D flip-flop, a Mod-60 counter and a shift register—and relate their operation to practical digital systems.

6.2 D flip-flop as a one-bit storage element

A D flip-flop captures the value at D on the active clock edge and retains that value until a later active edge. Multiple flip-flops form registers in processors and controllers. A clocked storage element can also be used in input-conditioning circuits, although switch debouncing normally requires additional timing or filtering logic.

ı debounce circuit that eliminates mechanical contact noise.



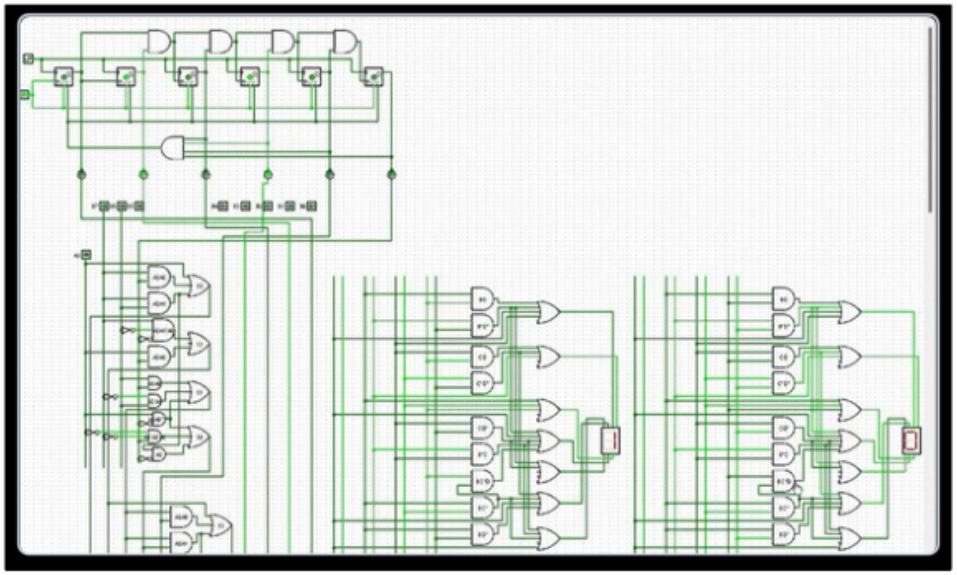
ıtion implementation of a 1-bit memory cell using a D Flip-Flop with 2:1 MUX

Figure 6.1. Source Logisim Evolution implementation of a D flip-flop storage cell.

CLOCK CONDITION	D INPUT	Q AFTER EVENT	MEANING
Active edge	0	0	A logic-low bit is stored
Active edge	1	1	A logic-high bit is stored
No active edge	X	Previous value	Stored information is retained

6.3 Mod-60 counter as a clock timebase

A Mod-60 counter cycles through sixty distinct values, from 0 to 59. Six binary flip-flops can represent the states. Terminal-count detection identifies the transition to 60 and causes the counter to return to zero. Such a stage is suitable for seconds or minutes counting.



t schematic of the Mod-60 counter with 6 synchronous flip-flops, BCD conversion l

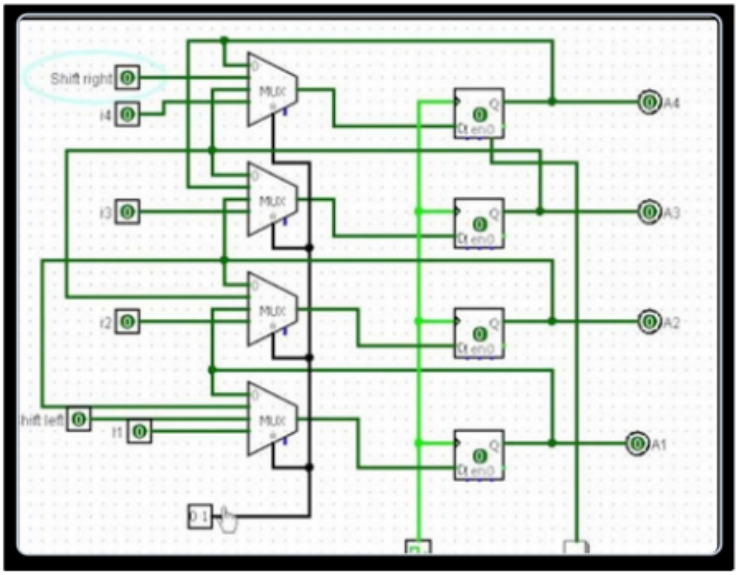
Figure 6.2. Source schematic of the Mod-60 counter and display logic.

The terminal state 60 is 111100■. A decode network detects this state and activates the reset/next-state logic so the valid sequence remains 000000■ through 111011■ (0 through 59).

COUNT	Q5 Q4 Q3 Q2 Q1 Q0	ACTION
0	000000	Initial state
1	000001	Normal increment
58	111010	One count before terminal
59	111011	Last valid count
60	111100	Reset/rollover to zero

6.4 Shift register for serial communication

A shift register moves stored bits one position per clock pulse. In Serial-In Parallel-Out (SIPO) operation, serial bits enter one at a time and become available together at the parallel outputs. Other modes can support parallel loading or shifting in the opposite direction.



n schematic of a 4-bit Universal Shift Register supporting Shift Rig

Figure 6.3. Source four-bit universal shift-register schematic.

CLOCK	SERIAL INPUT	Q3 Q2 Q1 Q0	OBSERVATION
Initial	—	0000	Register cleared
1	1	1000	First bit entered
2	0	0100	Next bit shifted in
3	1	1010	Third bit entered
4	1	1101	Four-bit word available

6.5 Application summary

The D flip-flop provides single-bit storage, the Mod-60 counter provides cyclic timing, and the shift register supports serial/parallel data movement. These structures are widely used in register files, timing circuits, serial interfaces and display drivers.

BLOCK	EXAMPLE APPLICATION	CORE FUNCTION
D flip-flop	CPU register / state storage	Stores one bit on a clock edge
Mod-60 counter	Clock seconds/minutes stage	Counts 60 states and rolls over
Shift register	UART/SPI data path / LED driver	Moves data between serial and parallel forms

6.6 Result and conclusion

The three sequential building blocks were studied through their state behaviour and simulation evidence. The experiment connects the abstract ideas of storage, counting and shifting with practical digital subsystems.

EXPERIMENT 7

Mini Automation System: Automatic Parking Barrier Control

Curriculum: Part C · Real-world applications **Course outcome:** CO2 **EDA environment:** Logisim Evolution

7.1 Abstract and objective

This experiment models an automatic parking entrance. A vehicle sensor detects an arriving car, a payment reader confirms authorization, and a barrier actuator opens or closes the gate. A safety response is included so that a vehicle detected during closing causes the barrier to reopen.

7.2 Inputs and outputs

The controller uses vehicle detection V, payment confirmation P, reset R and clock CLK. The output signals operate the barrier and indicate its status.

SIGNAL	TYPE	FUNCTION / LOGIC MEANING
V	Input	1 when a vehicle is detected
P	Input	1 when payment or ticket is validated
R	Input	1 requests a safe reset
CLK	Input	Clock event for synchronous state updates
OPEN	Output	1 commands barrier lifting
CLOSED	Output	1 indicates the barrier is down
RED / GREEN / YELLOW	Outputs	Waiting, proceed and closing-warning indicators

7.3 State model

Two flip-flops encode four controller states. The controller waits for a vehicle, checks payment, opens the barrier while the vehicle passes, and then closes once the detection area is clear.

STATE	Q1 Q0	CONDITION / TRANSITION	ACTION
S0 – Idle/closed	00	V=0 stays; V=1 → S1	Barrier down; wait for vehicle
S1 – Payment wait	01	V=1,P=0 stays; V=1,P=1 → S2; V=0 → S0	Wait for authorization
S2 – Passing	10	V=1 stays; V=0 → S3	Keep barrier open
S3 – Closing	11	V=0 → S0; V=1 → S2	Close if clear; reopen if obstructed

7.4 Next-state and output logic

The source assignment expresses the next-state logic using state terms S0–S3. The state terms are decoded from Q1 and Q0. Because reset and state encoding conventions can affect the final equations, the transition table should be treated as the primary specification during implementation.

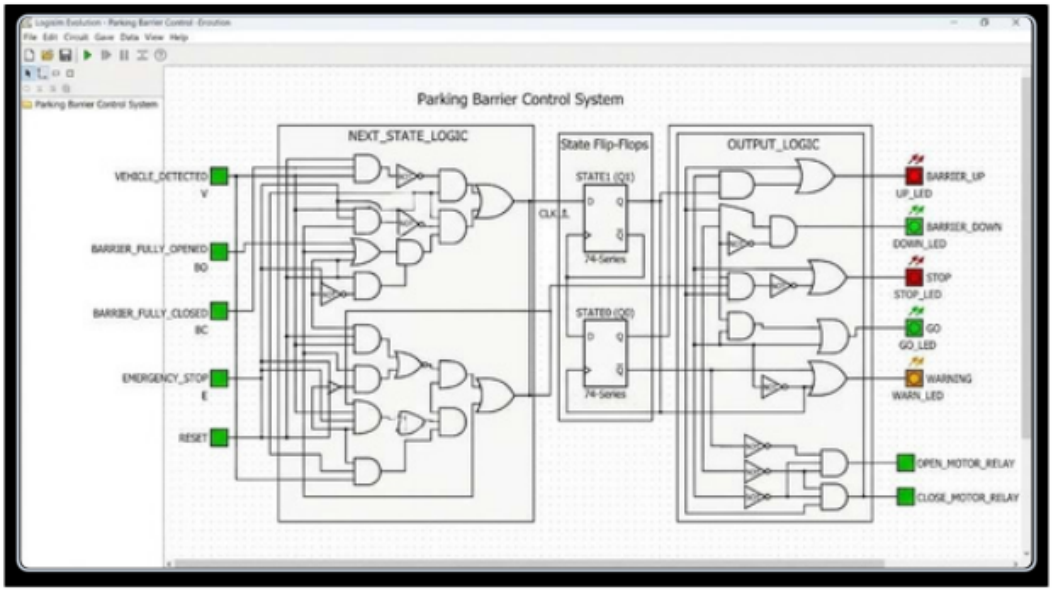
$$OPEN = Q1\ Q0'$$
$$GREEN = Q1\ Q0'$$
$$YELLOW = Q1\ Q0$$
$$RED = Q1'$$
$$CLOSED = Q1'$$

The source portfolio also records D1 = V(S1P + S2 + S3) and D0 = S0V + S1VP′ + S2V′. These equations should be checked against every row of the transition table before wiring, especially the reset and S3 recovery cases.

7.5 Verification scenarios

The test plan includes no vehicle, a detected vehicle without payment, valid payment, vehicle passage, vehicle departure, obstruction during closing, and emergency reset.

TEST	STATE / INPUT	EXPECTED RESPONSE
No vehicle	S0, V=0	Barrier closed; red indicator
Vehicle arrives	S0, V=1	Move to payment-wait state
Payment absent	S1, V=1,P=0	Remain closed and wait
Payment accepted	S1, V=1,P=1	Move to open state
Vehicle passing	S2, V=1	Keep barrier open
Area becomes clear	S2, V=0	Enter closing state
Obstacle reappears	S3, V=1	Return to open state
Reset	R=1	Return to safe idle state



olution schematic showing NEXT_STATE_LOGIC, State Flip-Flops (STATE1, STATE

Figure 7.1. Original Logisim Evolution schematic of the parking barrier controller.

7.6 Result and conclusion

The FSM captures the major phases of an automated parking gate and includes a reversal path when a vehicle is detected during closing. The simulation should be assessed against the transition table, with particular attention to safe reset behaviour and the obstacle-reopen condition.

EXPERIMENT 8

From Code to Circuit: 4-Bit Mini-ALU and Program Counter

Curriculum: Part C · Real-world applications **Course outcome:** CO4, CO1 **EDA environment:** Logisim Evolution

8.1 Aim and overview

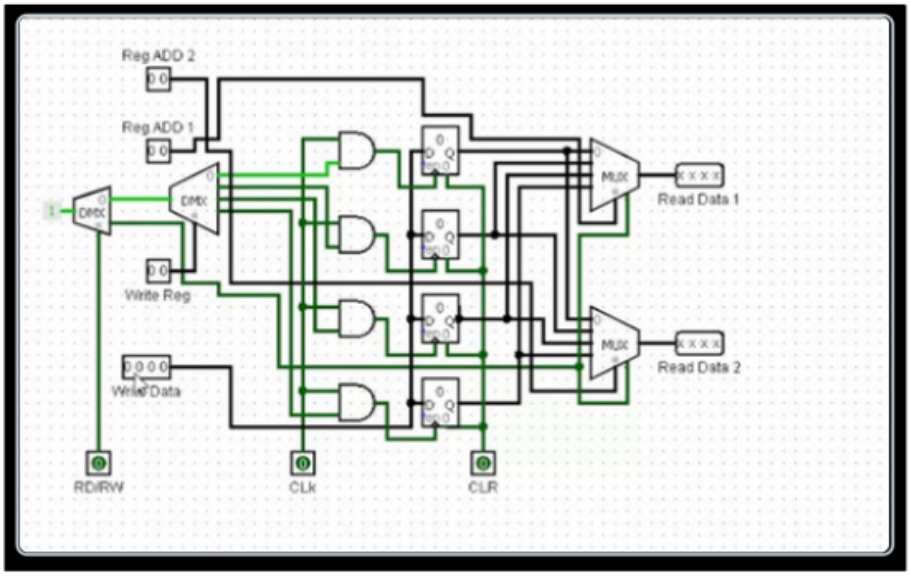
The design links a short software-level loop to two hardware structures: a four-bit arithmetic logic unit (ALU) and a four-bit synchronous program counter. The ALU performs arithmetic on operands, while the counter advances the instruction or iteration sequence.

```
for (int i = 0; i < 8; i++) {
    c = a + b;
}
```

8.2 Four-bit ALU datapath

The ALU receives two four-bit operands A and B and a mode control M. In addition mode, it computes A + B. In subtraction mode, the B input is complemented and a carry-in of 1 is applied, implementing two's-complement subtraction: A + B′ + 1. Registers and multiplexers provide operand selection and result write-back.

d registers and destination write-back.



lementation of the 4-bit register file and ALU datapath with Read Data 1, Rea

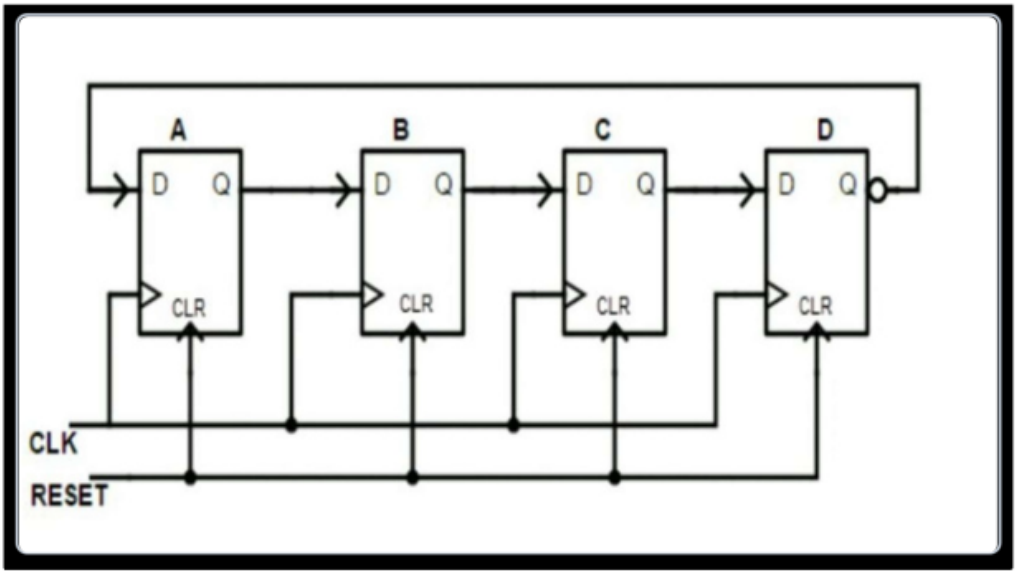
Figure 8.1. Source Logisim Evolution diagram of the register-file and ALU datapath.

A	B	M	OPERATION	RESULT	CARRY
0101 (5)	0011 (3)	0	5 + 3	1000 (8)	0
0110 (6)	0010 (2)	0	6 + 2	1000 (8)	0
0101 (5)	0011 (3)	1	5 – 3	0010 (2)	1
0111 (7)	0010 (2)	1	7 – 2	0101 (5)	1

8.3 Four-bit program counter

The program counter is made from four D flip-flops sharing a clock and reset. When enable is active, the counter increments by one on each clock edge. Reset initializes the count to 0000. A comparator can detect the terminal value

1000■, corresponding to eight iterations.



ution circuit schematic of the 4-bit synchronous Program Counter with flip-flops A, B, C

Figure 8.2. Source synchronous four-bit program-counter circuit.

8.4 Software-to-hardware mapping

A software variable that retains a value maps naturally to a register. An arithmetic operator maps to an ALU function. Assignment is represented by loading the result into a destination register. A loop counter can be represented by a counter, while the loop condition can be implemented using a comparator and control logic.

SOFTWARE CONSTRUCT	HARDWARE REPRESENTATION	ROLE
Variable a	Register A	Stores first operand
Variable b	Register B	Stores second operand
c = a + b	ALU + destination register	Computes and stores result
i++	Program counter / incrementer	Advances iteration state
i < 8	Magnitude comparator	Determines whether execution continues

8.5 Simulation procedure and trace

The ALU should be tested in both modes with several operand pairs. The PC should be reset, enabled and clocked while its output is observed. The sequence reaches 1000■ after eight increments; the comparator then indicates that the loop condition is no longer true.

EVENT	PC	ITERATION VALUE	i < 8	EXPECTED ACTION
Reset	0000	0	True	Initialize registers
Clock 1	0001	1	True	Perform addition
Clock 2	0010	2	True	Perform addition
…	…	…	…	Continue loop
Clock 7	0111	7	True	Final iteration
Clock 8	1000	8	False	Stop loop

8.6 Result and conclusion

The ALU demonstrates arithmetic datapath operations, while the PC illustrates clocked sequencing and loop progression. Together, these blocks show how high-level arithmetic and repetition can be represented by registers, combinational logic, counters and comparison circuitry.

EXPERIMENT 9

AI-Assisted Code-to-Gate Explainer

Curriculum: Part C · Real-world applications **Course outcome:** CO4, CO2 **EDA environment:** Logisim Evolution

9.1 Objective

To interpret a conditional statement and identify a corresponding digital architecture. The circuit must output the larger of two two-bit unsigned values.

9.2 Code under analysis

The source program compares A and B. If A is greater, C receives A; otherwise, C receives B. Therefore, the expected output is the maximum of the two input values.

```
if (A > B)
  C = A;
else
  C = B;
```

9.3 Hardware components

A two-bit magnitude comparator evaluates A > B. Its output controls two 2:1 multiplexers: one selects the most significant output bit C1, and the other selects the least significant output bit C0. Both multiplexers use the same select signal.

SEL = 1 when A > B; otherwise SEL = 0.
C = A when SEL = 1; C = B when SEL = 0.

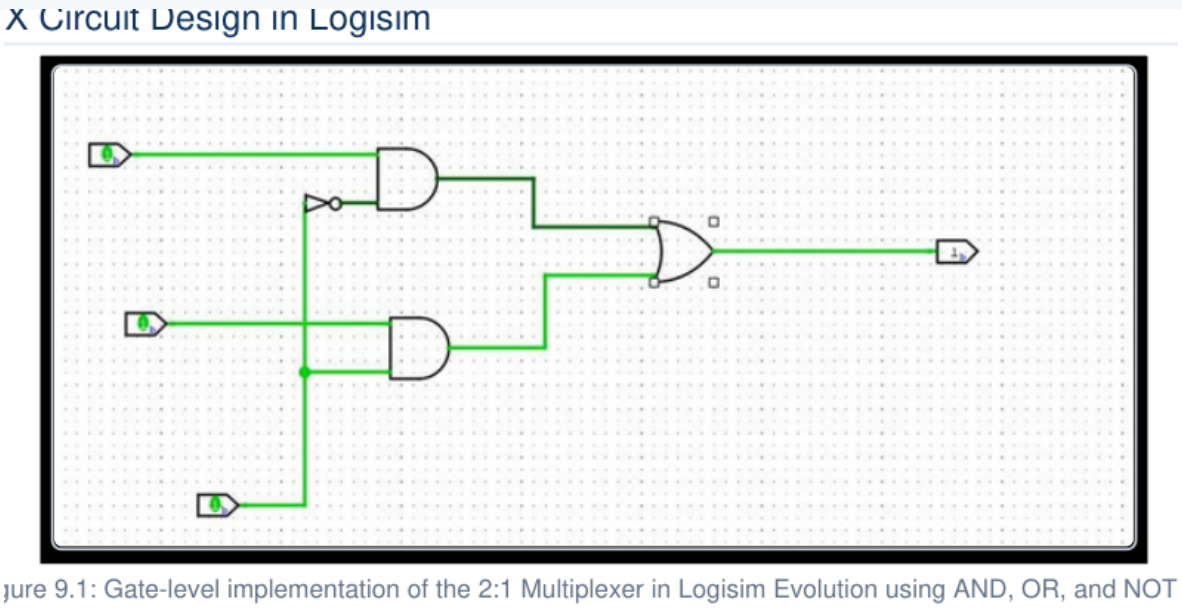


Figure 9.1. Source gate-level implementation of a two-to-one multiplexer.

s all 16 binary combinations o



5. Truth Table and Verification

A	B	A > B	C_1	C_0	C (Decimal)
00	00	0	0	0	0
00	01	0	0	1	1
00	10	0	1	0	2
00	11	0	1	1	3
01	00	1	0	1	1
01	01	0	0	1	1
01	10	0	1	0	2
01	11	0	1	1	3
10	00	1	1	0	2
10	01	1	1	0	2
10	10	0	1	0	2
10	11	0	1	1	3
11	00	1	1	1	3
11	01	1	1	1	3
11	10	1	1	1	3
11	11	0	1	1	3

ıre output C equals the maximum of in

Figure 9.2. Source truth-table simulation view for the two-bit maximum circuit.

9.4 Verification strategy

Each input A and B can take one of four values: 00, 01, 10 or 11. The complete test set therefore contains sixteen input pairs. For every pair, the output should equal the numerically larger operand. When A and B are equal, either multiplexer input produces the same result; the stated design selects B in the equality case.

A	B	EXPECTED C = max(A,B)	CHECK
00	00	00	Equal inputs
00	01	01	B is larger
01	00	01	A is larger
01	10	10	B is larger
10	01	10	A is larger
10	11	11	B is larger
11	10	11	A is larger
11	11	11	Equal inputs

9.5 AI analysis and engineering considerations

The AI-assisted explanation identifies a comparator-plus-multiplexer datapath as the hardware equivalent of the conditional assignment. A practical synchronous implementation may add input/output registers if the surrounding system requires stable clocked interfaces. The comparator output should be checked for timing and used consistently as the select signal for both bit slices.

9.6 Result and conclusion

The conditional program structure maps to a comparator that generates a control decision and a pair of multiplexers that route the selected operand to C. The truth-table simulation provides a direct check that the circuit implements max(A,B) over the tested input combinations.

OVERALL CONCLUSION

The nine experiments cover a progression from combinational Boolean design to sequential control and datapath architecture. K-map minimization and natural-language translation demonstrate ways to derive logic efficiently. FSM-based controllers show how state and timing govern real-world behaviour. Fault diagnosis emphasizes comparison against expected truth-table behaviour. Counters, flip-flops and shift registers provide the storage and sequencing structures used in digital systems. The ALU, program counter and comparator–multiplexer design connect software-level operations to hardware implementation.

Across the portfolio, simulation is used as a verification step rather than as a substitute for reasoning. Each circuit should be checked against its specification, including boundary cases, unused states, reset behaviour and all relevant input combinations. AI assistance is most useful when its proposed equations and interpretations are independently validated.

EDITORIAL AND VERIFICATION NOTES

This report is a rewritten and professionally typeset study document based on the uploaded ESC392 portfolio. The original figures and circuit screenshots have been retained as supporting illustrations. The source PDF contains a few malformed symbols and inconsistencies; the following clarifications are made explicit in this edition:

• Experiment 4: the supplied faulty Sum vector corresponds to A ⊕ B, not A·B. The diagnosis and corrected equation follow the vector itself.
• Experiment 5: the displayed increment equations are expressed in standard XOR toggle form; terminal-count decoding is still required for Mod-60 and Mod-24 behaviour.
• Experiment 7: the source next-state equations should be validated row-by-row against the transition table and reset behaviour before physical implementation.
• Experiment 8: corrupted subtraction symbols are interpreted as subtraction implemented using two’s complement.

STUDENT DETAILS

FIELD	DETAIL
Student	
Roll number	523
Section	G
Course	ESC392 – Digital Electronics Laboratory
Academic session	2026

END OF REPORT